Computer vision and deep learning

Lecture 2

Today’s agenda

- Linear classifiers
- Evaluating a classifier
- Loss functions
- Gradient descent (to be continued)

Image classification I

Classifier

CAT*

$\hat{y}_i$ - prediction

X_i - image sample to be classifier

y_i - ground truth

* assuming that we have a set of labels L = {cat, shark, ship …}

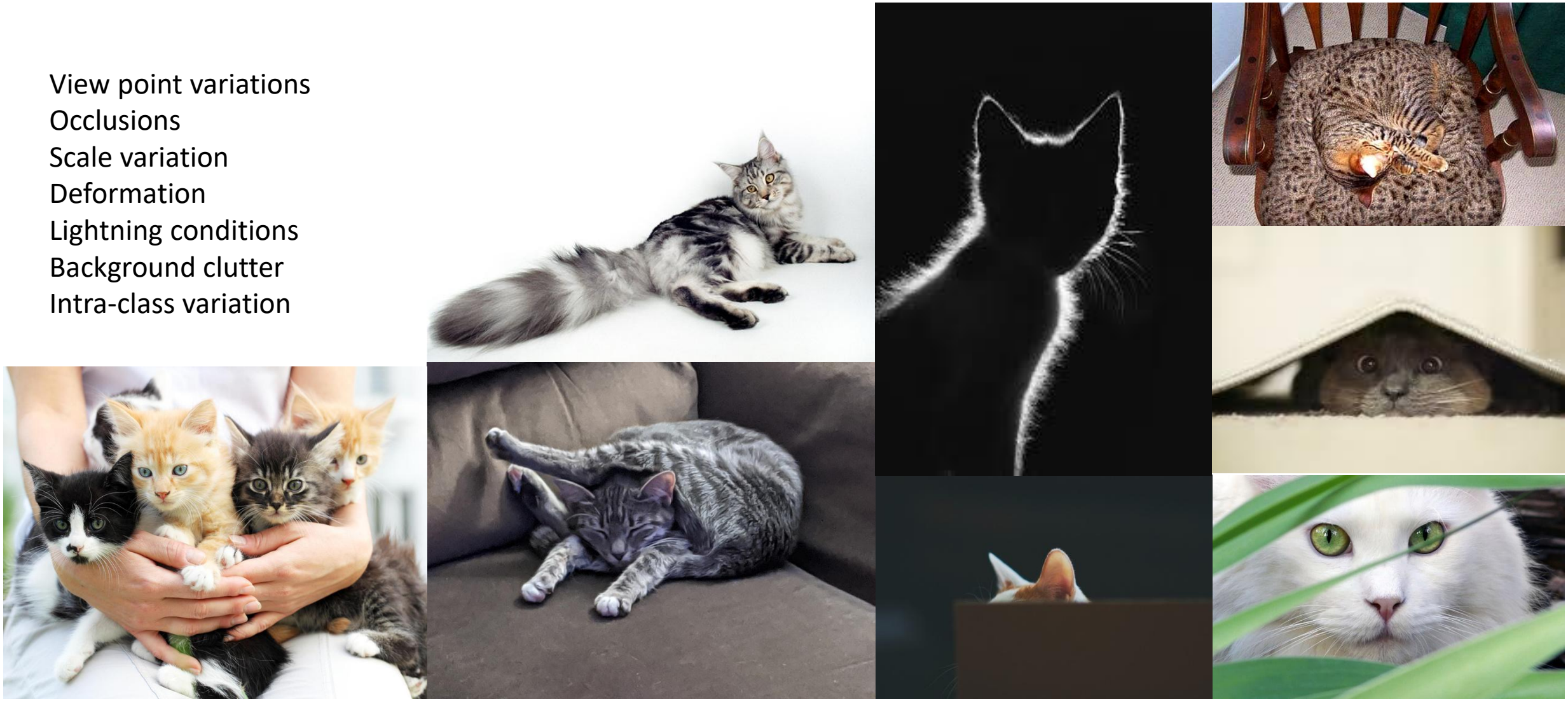
Image source: https://katzenworld.co.uk/2019/06/30/mews-funny-cat-picture-collection/

Image classification II

- Unlike other computer science problems:
  - Find the maximum value in an array
  - Sort an array
  - Find the shortest distance between two nodes in a graph
- **No obvious way** to implement an algorithm for image classification

Classification challenges

View point variations
Occlusions
Scale variation
Deformation
Lightning conditions
Background clutter
Intra-class variation



Linear classification

Linear classification

- Define a function which maps image pixels to class scores

$$f(x_i, W, b) = W\, x_i + b$$

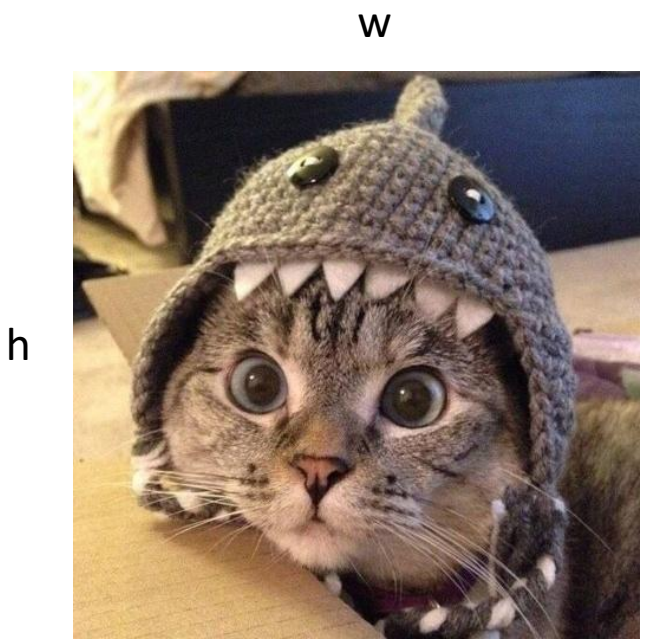
weights

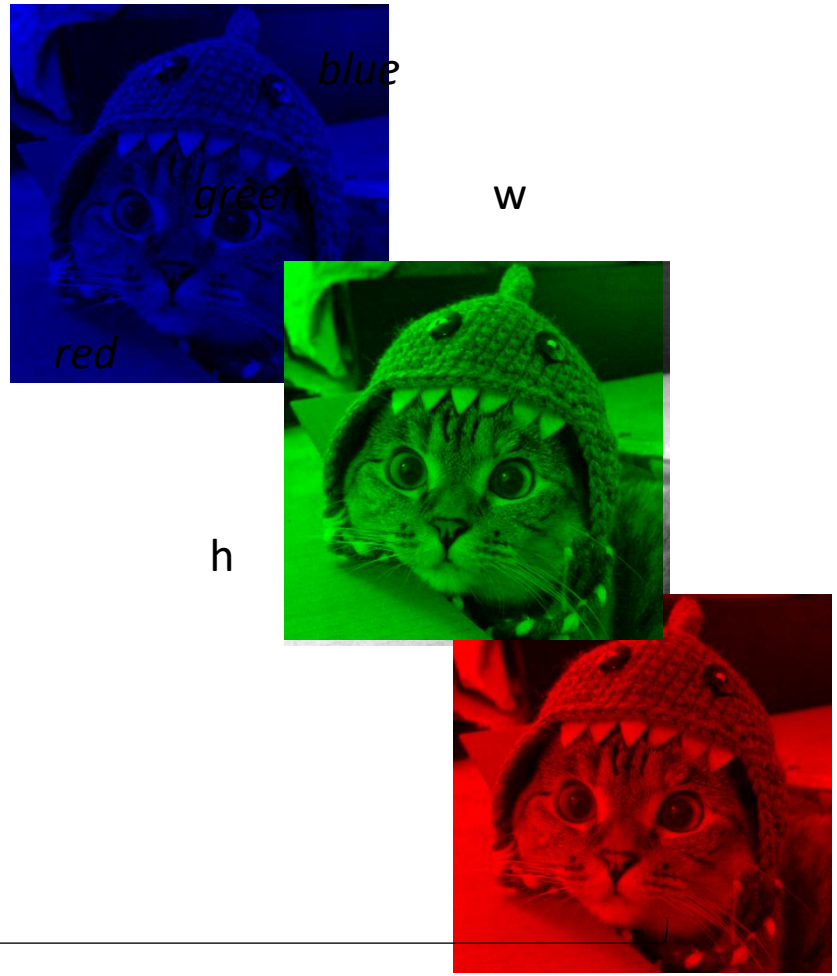
bias

Classifier
$f(x_i, W, b)$

N class scores

Image representation





1D vector with w*h*3 values between [0, 255]

Flatten the image to a 1D array

Linear classification

CIFAR 10 example:

- 3 channel images, 32x32 pixels $x_i \in \mathbb{R}^D$
- N = 10 class scores

$$f\colon \mathbb{R}^D \rightarrow \mathbb{R}^N \qquad , f(x_i, W, b) = W \cdot x_i + b$$

Classifier
$f(x_i, W, b)$

N class scores

N = 10

D = 32x32x3 = 3072

Linear classification

CIFAR 10 example:

- 3 channel images, 32x32 pixels, x_i size [3072 x 1]
- N = 10 class scores

$x_i \in \mathbb{R}^D$

$$f(x_i, W, b) = \underset{???}{W} \underset{[3072 \times 1]}{\boxed{x_i}} + \underset{[D \times 1]}{b}$$

$f: \mathbb{R}^D \rightarrow \mathbb{R}^N$

Classifier
$f(x_i, W, b)$

N class scores

N = 10

D = 32x32x3 = 3072

Linear classification

CIFAR 10 example:

- 3 channel images, 32x32 pixels
- N = 10 class scores

$x_i \in \mathbb{R}^D$

[D x 1]

size: N x D

$$f(x_i, W, b) = \boxed{W} x_i + b$$

10 x 3072

[3072 x 1]

$f: \mathbb{R}^D \rightarrow \mathbb{R}^N$

Classifier
$f(x_i, W, b)$

N class scores

N = 10

D = 32x32x3 = 3072

Linear classification

CIFAR 10 example:

- 3 channel images, 32x32 pixels
- N = 10 class scores

$x_i \in \mathbb{R}^D$

[D x 1]

size: N x D

size: N x 1

$$f(x_i, W, b) = \boxed{W}\, x_i + \boxed{b}$$

$f: \mathbb{R}^D \rightarrow \mathbb{R}^N$

10 x 3072

10 x 1

[3072 x 1]

Classifier

$f(x_i, W, b)$

N class scores

N = 10

D = 32x32x3 = 3072

Linear classification - simplified example

weights W

0.7	-0.3	2.5	1
-0.3	0.5	3.1	-3
0	0.25	0.4	0.1

.

X

57
77
200
128

+

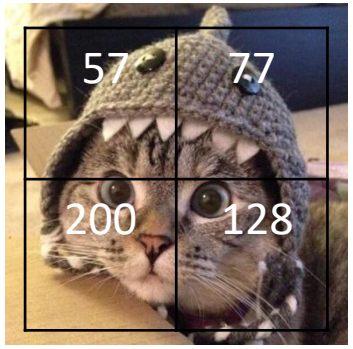
bias b

1.7
-2
3

=

646.5	cat score
255.4	boat score
115.05	shark score

Flattened input image



Example adapted from: https://cs231n.github.io/

Linear classification – implementation hints

The bias trick

weights W

0.7	-0.3	2.5	1	
-0.3	0.5	3.1	-3	
0	0.25	0.4	0.1	

.

X

57
77
200
128

+

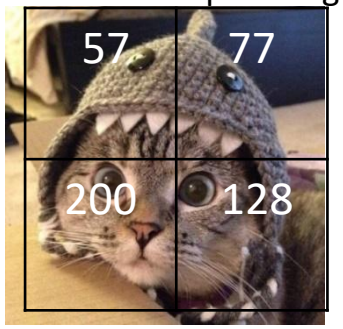
bias b

1.7
-2
3

=

646.5	cat score
255.4	boat score
115.05	shark score

Flattened input image



Example adapted from: https://cs231n.github.io/

Linear classification – implementation hints

The bias trick

weights W

0.7	-0.3	2.5	1	1.7
-0.3	0.5	3.1	-3	-2
0	0.25	0.4	0.1	3

bias b

·

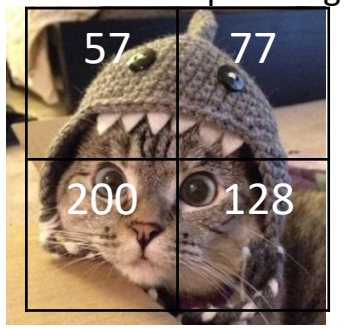
x

57
77
200
128
1

=

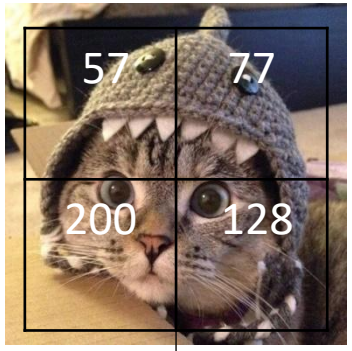
646.5	cat score
255.4	boat score
115.05	shark score

Flattened input image



Example adapted from: https://cs231n.github.io/

Linear classification - simplified example



W

0.7	-0.3
2.5	1

-0.3	0.5
3.1	-3

0 57	0.25
0.4	0.14

b

1.7

-2

3

score

646.5

255.4

115.05

0.7	-0.3	2.5	1
-0.3	0.5	3.1	-3
0	0.25	0.4	0.1

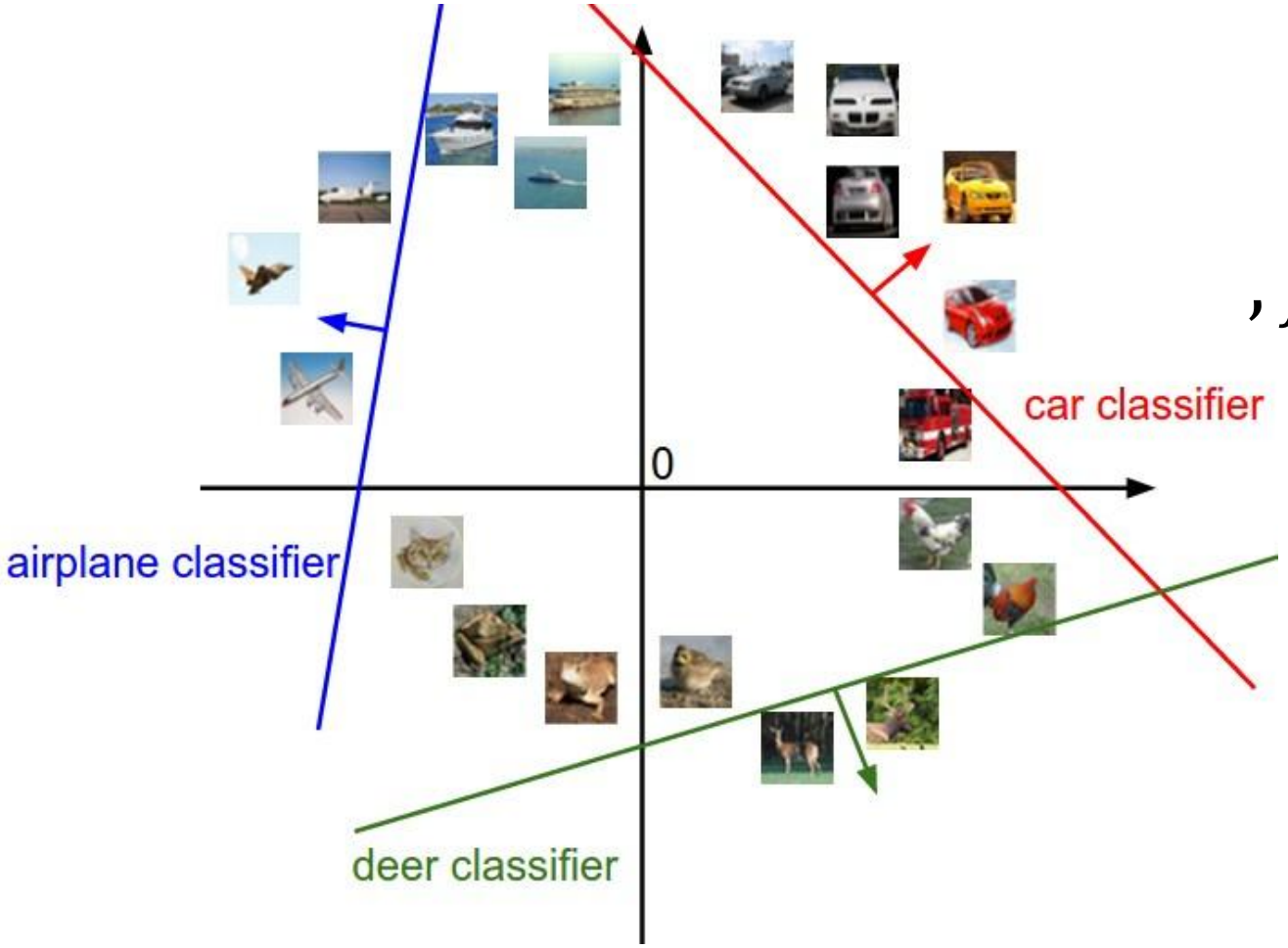
Example adapted from: https://cs231n.github.io/

Linear classification – visual interpretation

- Linear classification can be seen as a *template matching* process, where the weights are learned
- Each row of the weights matrix is a *prototype* (template) for one of the classes

Image source: https://cs231n.github.io/

Linear classification – geometric interpretation



Hyper-plane cutting up the space

$$, f(x_i, W, b) = W \ x_i + b$$

Image source: https://cs231n.github.io/

Linear classification

$$f(x_i, W, b) = W\, x_i + b$$

- Input x_i → fixed
- N: number of classes →fixed
- W – weights → we have control over their setting
- b – bias → we have control over their setting
  - b doesn't actually interact with the data
- *f* – ***score function*** *- map raw pixels to class scores*

Goal: set W and b such that the correct class has a higher value than in incorrect ones

Next steps

- Define a **loss function *L***
  - ***L*** **-** measures the quality of ***W*** and ***b*** based on how well the predicted scores agree with the ground truth labels
  - ***L*** **–** how happy are we with the scores across the training data? How good is our (current) classifier?
- Find an efficient way to determine the parameters (W and b) such that the loss function is minimized

Loss function

$$L = \frac{1}{N}\sum_{i}^{N} L_i\,(f(x_i\,, W), y_i)$$

- N – number of training examples
- x_i - ith image in our training dataset
- y_i - ground truth label for the ith image in our training set (i.e. cat, boat or shark)

Example – softmax classifier

Class	
cat	3.2
boat	-5
shark	10

We don’t have interpretation for the outputs of the linear layer

What if we would like to determine the probability of each class?

Example – softmax classifier

We don't have interpretation for the outputs of the linear layer

What if we would like to determine the probability of each class?

Class	
cat	3.2
boat	-5
shark	10

$$s = f(x_i, W)$$

Softmax function

$$P(Y = c \mid X = x_i) = \frac{e^{s_c}}{\sum_j e^{s_j}}$$

Example – softmax classifier

So far, we don't have interpretation for the outputs of the score function

What if we would like to determine the probability of each class?

Class	
cat	3.2
boat	-5
shark	10

$$s = f(x_i, W)$$

Softmax function

$$P(Y = c \mid X = x_i) = \frac{e^{s_c}}{\sum_j e^{s_j}}$$

normalize

Class	
cat	3.2
boat	-5
shark	10

s =
[
3.2
-5
10
]

e^s =
[
24.5325
0.0067
22026.4658
]

$\sum_j e^{s_j}$ = 22026.4658

$$\frac{e^{s_c}}{\sum_j e^{s_j}} =$$

[
0.0011125
0.0000003
0.9988872
]

softmax function

0 – cat class label, 1 – boat, 2 – shark class

softmax function

[
0.0011125
0.0000003
0.9988872
]

vs

hardmax function, one hot encoding

[
1
0
0
]

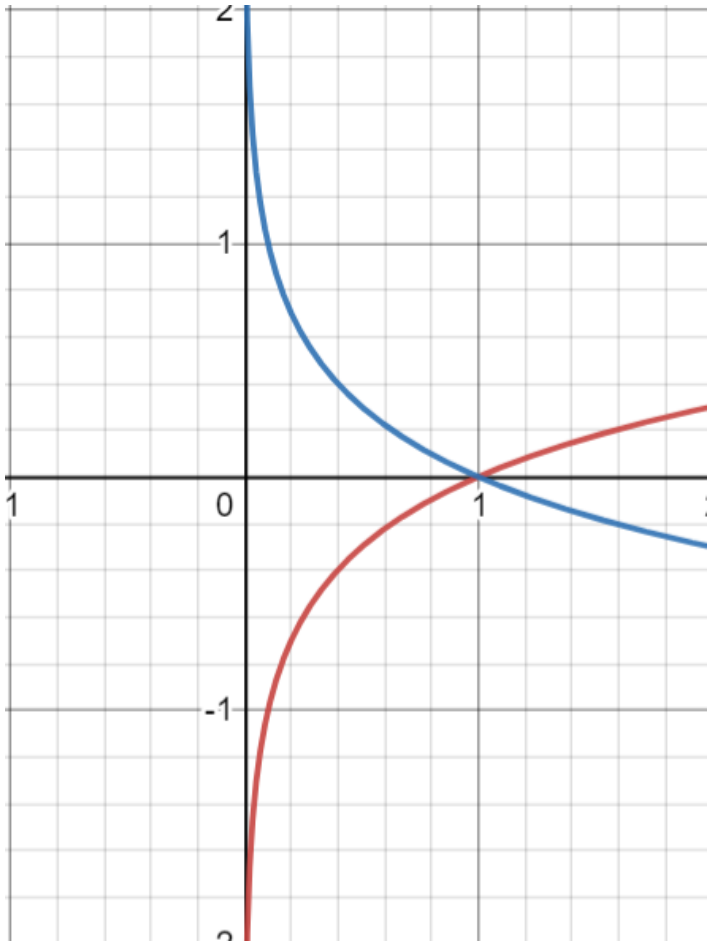
Cross entropy loss-function

s_j - prediction for class j

$$L_i = -\sum_{j=1}^{C} y_j \log(s_j)$$

$$= -y_c \log(s_c)$$

$$= -\log(s_c)$$



$\log x$
$-\log x$

$$L_i = -\sum_{j=1}^{C} y_j \log(s_j)$$

s_j - prediction for class j

$$= -y_c \log(s_c)$$

$$= -\log(s_c)$$

Intuition:

- The loss $L_i = -\log(s_c)$ should be as small as possible
  → the correct probability class should be as big as possible

[
1
0
0
]

[
0.0011125
0.0000003
0.9988872
]

$$L_i = -\sum_{j=1}^{C} y_j \log(s_j)$$

$$= -y_c \log(s_c)$$

$$= -\log(s_c)$$

s_j - prediction for class j

Intuition:

- The loss $L_i = -\log(s_c)$ should be as small as possible
  → the correct probability class should be as big as possible

This loss function just looks at the correct class from the ground truth and tries to make the predicted probability of that class as big as possible

$$L_i = -\log(s_c)$$

$$L = \frac{1}{N}\sum_{i}^{N} L_i$$

- What is the minimum and maximum value of L_i?

$$L_i = -\log(s_c)$$

$$L = \frac{1}{N}\sum_{i}^{N} L_i$$

- What is the minimum and maximum value of L_i?
  - Min 0
  - Max infinity

Example – focal loss

- Problems with cross-entropy loss
  - **Class imbalance is a problem:** majority class examples will dominate the loss function and gradient descent, causing the weights to update in the direction of the model becoming more confident in predicting the majority class while putting less emphasis on the minority classes
  - Fails to **distinguish** between **hard** and **easy examples**.

Example – Balanced CE loss

- **Balanced Cross-Entropy Loss (BlCE) –** add a weighing factor to each class α_j
  - could be the inverse class frequency or a hyper-parameter that is determined by cross-validation

$$CE = -\sum_{j=1}^{C} y_j \log(s_j)$$

$$\mathrm{B}lCE = -\sum_{j=1}^{C} y_j \cdot \alpha_j \log(s_j)$$

  - **Solves class imbalance problem**

Example – Focal loss

- Focuses on the examples that the model gets wrong rather than the ones that it can confidently predict
- Ensures predictions on hard examples improve over time rather than becoming overly confident with easy ones.
- **Down weighting**
  - reduce the influence of easy examples on the loss function, resulting in more attention being paid to hard examples.

Example – focal loss

- **Down weighting**
  - <u>reduce the influence of easy examples</u> on the loss function, resulting in more attention being paid to hard examples.
  - γ – **focusing** factor; hyperparameter
  - $(1 - p_t)^\gamma$ - **modulating** factor

$$CE = -\sum_{j=1}^{C} y_j \log(s_j)$$

$$FL = -\sum_{j=1}^{C} y_j \cdot (1 - s_j)^\gamma \log(s_j)$$

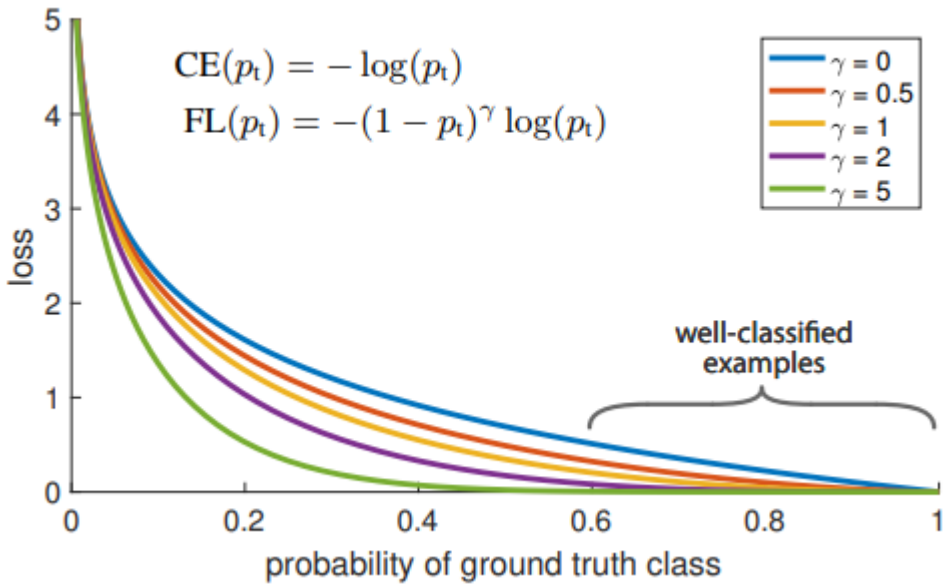
$$FL = -p_t(1 - p_t)^\gamma \log(p_t)$$

p_t - correct class prediction

Example – focal loss

$$FL = -(1 - p_t)^{\gamma} \log(p_t)$$

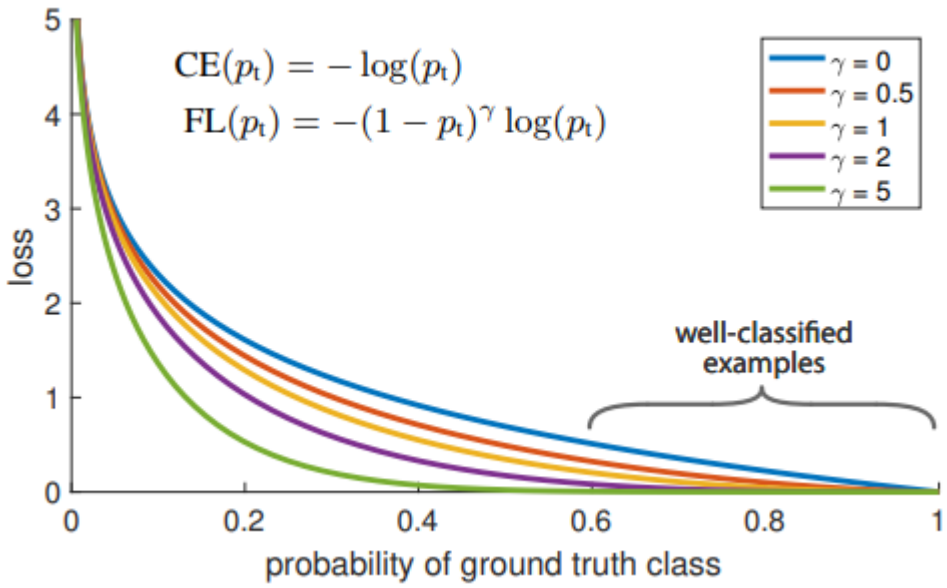
- Misclassification: p_t - ?



Example – focal loss

$$FL = -(1 - p_t)^{\gamma} \log(p_t)$$

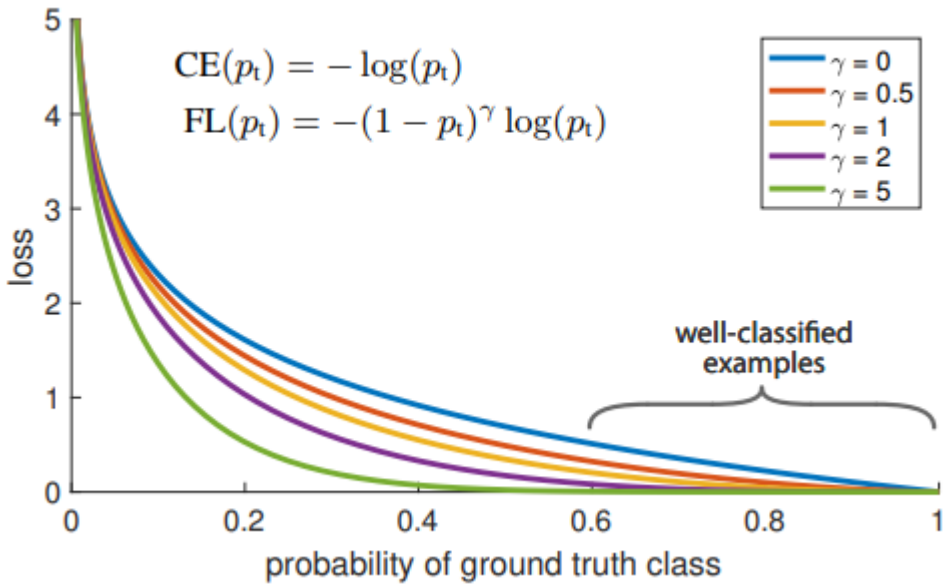
- Misclassification: p_t - small → $(1 - p_t)^{\gamma} \approx 1$ → behaves like CE
- Correct and confident prediction ?



Example – focal loss

$$FL = -(1 - p_t)^{\gamma} \log(p_t)$$

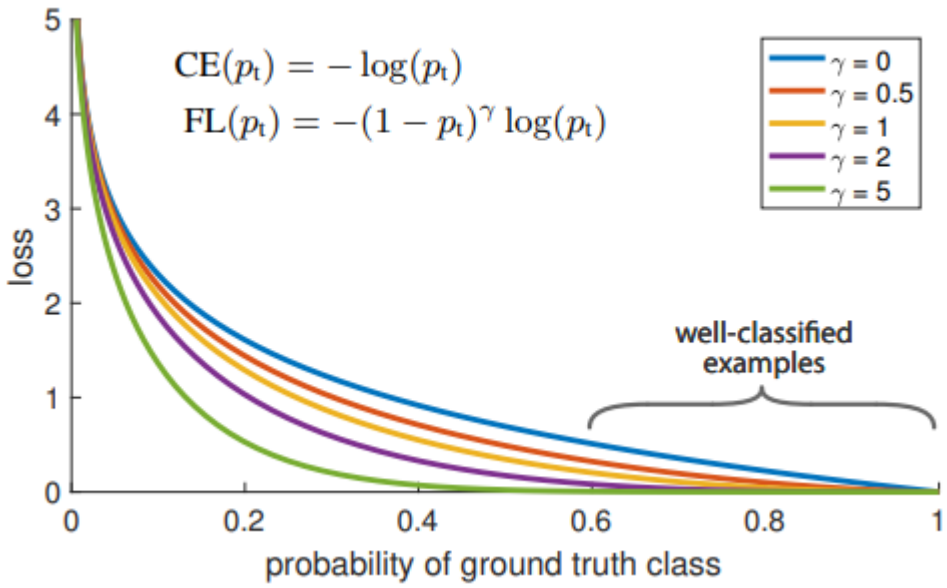
- Misclassification: p_t - small → $(1 - p_t)^{\gamma} \approx 1$ → behaves like CE
- Correct and confident prediction: $p_t \approx 1$ → $(1 - p_t)^{\gamma} \approx 0$ → down-weighting the loss value for well-classified examples



Example – focal loss

$$FL = -(1 - p_t)^{\gamma} \log(p_t)$$

- $\gamma \geq 1$ rescales the modulating factor such that the easy examples are down-weighted more than the hard ones
- e.g. $\gamma = 2$
  - $p_1 = 0.6$: CE: 0.2218, FL = 0.035488; 6.25 decrease
  - $p_2 = 0.9$: CE = 0.04575, FL = 0.0004575; 100x decrease



Example – focal loss

- In practice, focal loss is combined it balanced cross entropy loss by adding also the **weighing factor** α_t

$$FL = -\alpha_t(1 - p_t)^{\gamma}\log(p_t)$$

Example – hinge loss or SVM loss

- $s_j = f\left(x_j\ , W\right)$ - the prediction of the jth image in our training dataset
- m – margin

$$L_i\ =\ \sum_{j\ !=y_i} \max(0, s_j - s_{y_i} + m)$$

Example – hinge loss

- $s_j = f\left(x_j\ , W\right)$ - the prediction of the jth image in our training dataset
- m – margin

$$L_{\ i} = \sum_{j\ !=y_i} \max(0, s_j - s_{y_i} + m)$$

$\max(0, s_j - s_{y_i} + m)$

if $s_{y_i} \geq\ s_j + m$

return 0

else:

return $s_j - s_{y_i} + m$

Example – hinge loss

- $s_j = f\left(x_j\ , W\right)$ - the prediction of the jth image in our training dataset
- m – margin

$$L_{\ i} = \sum_{j\ !=y_i} \max(0, s_j - s_{y_i} + m)$$

$$= \sum_{j\ !=y_i} \begin{cases} 0, & s_{y_i} \geq s_j + m \\ m + \ s_j \ - s_{y_i}, & otherwise \end{cases}$$

$\max(0, s_j - s_{y_i} + m)$

```
if s_{y_i} ≥ s_j + m
    return 0
else:
    return s_j − s_{y_i} + m
```

Example – hinge loss

- $s_j = f\left(x_j, W\right)$ - the prediction of the jth image in our training dataset
- m – margin

$$L_i = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + m)$$

$$= \sum_{j\,!=y_i} \begin{cases} 0, & s_{y_i} \geq s_j + m \\ m + s_j - s_{y_i}, & otherwise \end{cases}$$

$\max(0, s_j - s_{y_i} + m)$

if $s_{y_i} \geq s_j + m$
    return 0
else:
    return $s_j - s_{y_i} + m$

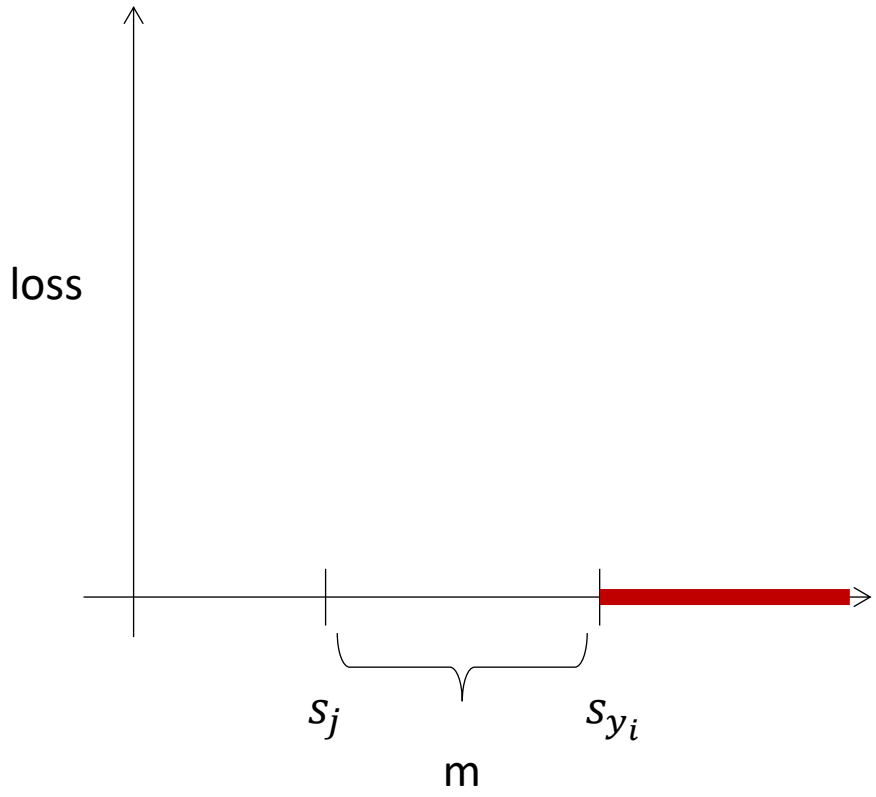
Intuition: we are "happy" if the score of the correct class if larger, by a margin of at least m, than the score of the other classes

Example – hinge loss

- $s_j = f\left(x_j\ , W\right)$ - the prediction of the jth image in our training dataset
- m – margin

$$L_{\ i} = \sum_{j\ !=y_i} \max(0, s_j - s_{y_i} + m)$$

$$= \sum_{j\ !=y_i} \begin{cases} 0, & s_{y_i} \geq s_j + m \\ m + \ s_j \ - s_{y_i}, & otherwise \end{cases}$$

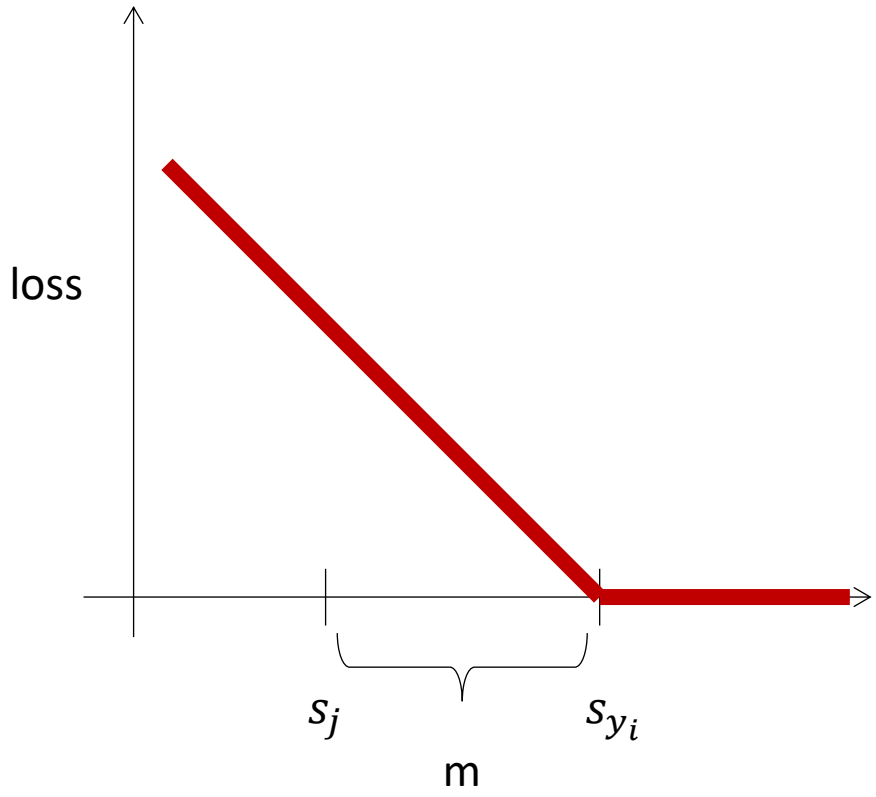


Example – hinge loss

- $s_j = f\left(x_j, W\right)$ - the prediction of the jth image in our training dataset
- m – margin

$$L_i = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + m)$$

$$= \sum_{j\,!=y_i} \begin{cases} 0, & s_{y_i} \geq s_j + m \\ s_j - s_{y_i}, & \underline{otherwise} \end{cases}$$



Example – hinge loss

Class			
cat	3.2	0.1	0.3
boat	-5	14	8
shark	10	5	14
Hinge loss			

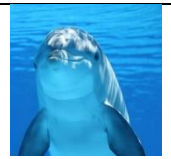
$$L_{H_i} = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + m)$$

Let's set m = 1

For image 1

=max(0, - 5 – 3.2 + 1) + max(0, 10 – 3.2 + 1)
= 0 + 7.8
= 7.8

Example – hinge loss

Class			
cat	3.2	0.1	0.3
boat	-5	14	8
shark	10	5	14
Hinge loss	**7.8**		

$$L_{H_i} = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + m)$$

Let's set m = 1

For image 1

=max(0, - 5 – 3.2 + 1) + max(0, 10 – 3.2 + 1)
= 0 + 7.8
= 7.8

Example – hinge loss

Class			
cat	3.2	0.1	0.3
boat	-5	14	8
shark	10	5	14
Hinge loss	**7.8**	**0**	**0**

Total loss L

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i$$

L = (7.8 + 0 + 0)/3 = 2.6

$$L_{H_i} = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + m)$$

Let's set m = 1

For image 2

= max(0, 0.1 – 14 + 1) + max(0, 5 – 14 + 1)
= 0 + 0
= 0

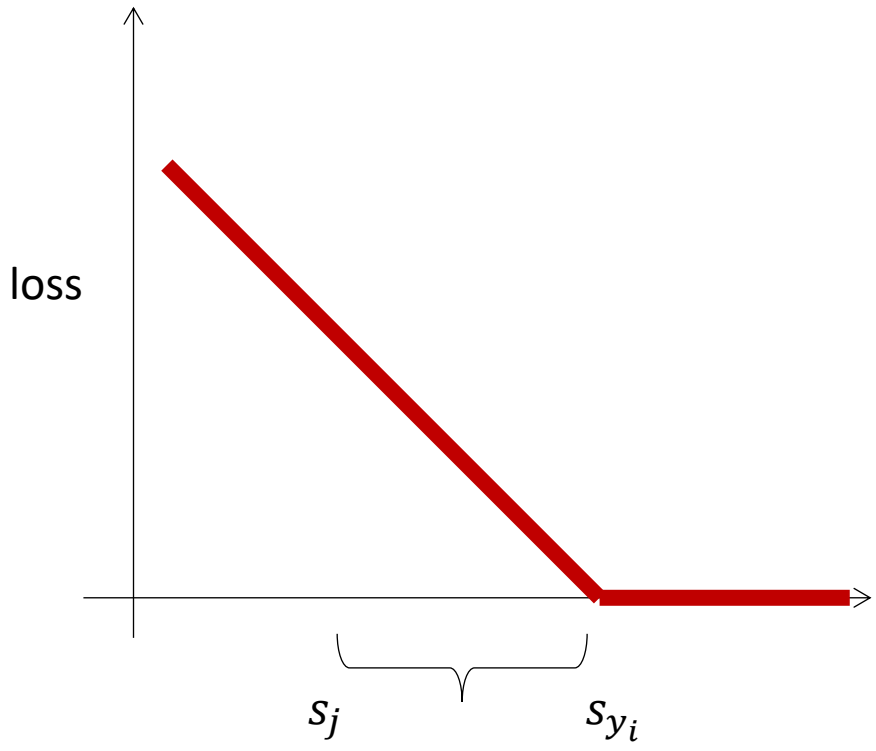
For image 3

= max(0, 0.3 – 14 + 1) + max(0, 8 – 14 + 1)
= 0 + 0
= 0

Hinge loss

- What is the min value and the max value of hinge loss?

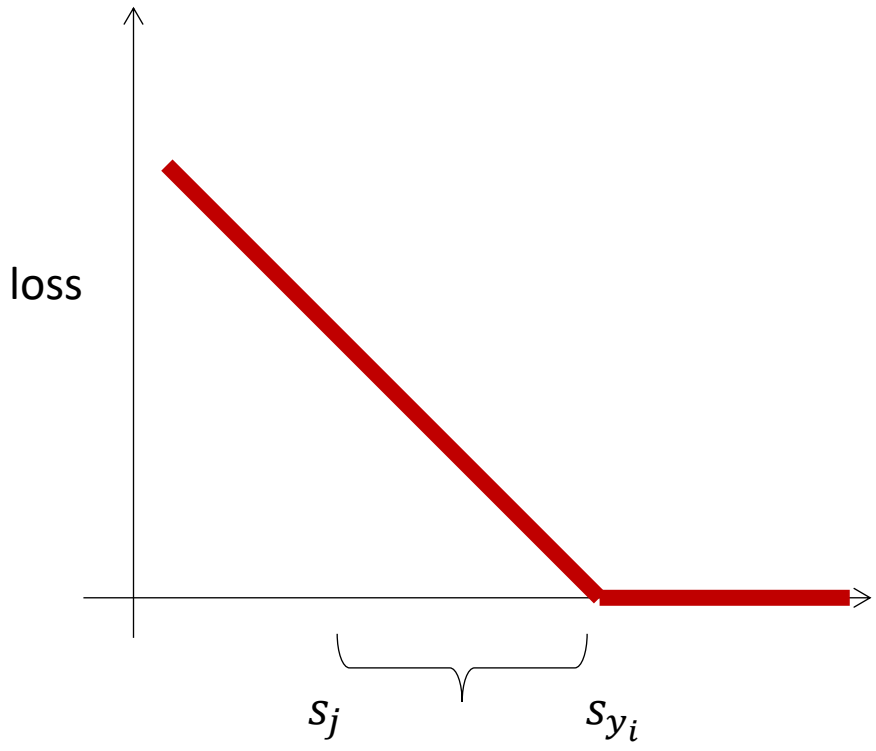
$$L_i = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + 1)$$



Hinge loss

- What is the min value and the max value of hinge loss?
  - min is 0: the correct score is much larger (>=m) than the incorrect scores
  - max is infinity
- In the beginning of the training (W ≈ 0) so all scores ≈ 0. What is the value of L ?

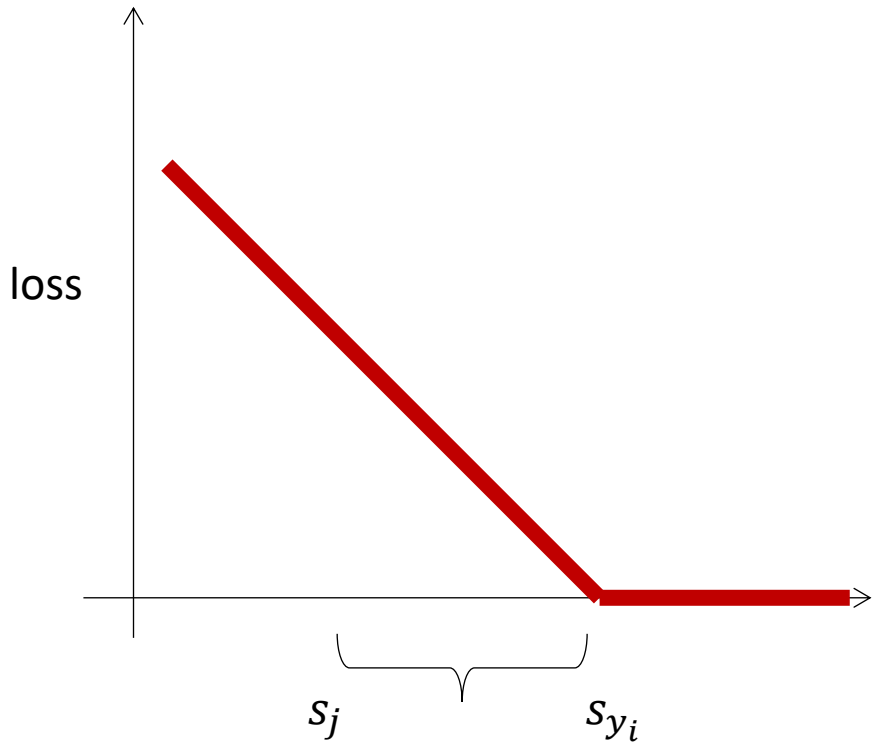
$$L_{H\,i} = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + 1)$$



Hinge loss

- What is the min value and the max value of hinge loss?
- In the beginning of the training (W ≈ 0) so all scores ≈ 0. What is the value of L ?
  - C – 1

$$L_{H\,i} = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + 1)$$



Regularization

- Suppose that you found some values for the parameters W such that the loss function is 0. Do you think there is a single setting for the parameters W, such that the loss is 0?

Regularization

- Suppose that you found some values for the parameters W such that the loss function is 0. Do you think there is a single setting for the parameters W, such that the loss is 0?
- W, 2W, 3W, αW
- extend the loss function with a **regularization penalty** to discourage large weights

Regularization

Regularization loss

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i + \lambda R(W)$$

Data loss

- Prevent overfitting (the model will perform ”too” well on the training data)
- Occam’s razor: William of Ockham (c. 1287–1347)
- Make the model simpler

Occam’s razor: "the simplest explanation is most likely the right one"

Image source: https://ro.pinterest.com/pin/192669690286459858/?nic_v2=1a57hjLPe

Regularization

- L2 regularization
  - $R(W) = \sum_r \sum_c W_{r,c}^2$

x = [2, 0.1, 0.1, 15]
W1 = [0.2, 0.8, 0.2, 0.1] → R(W) = 0.73
W2 = [1, 0, 0, 0] → R(W) = 1

W1· X = 2
W2· X = 2

Regularization

- L2 regularization
  - $R(W) = \sum_r \sum_c W_{r,c}^2$

x = [2, 0.1, 0.1, 15]
W1 = [0.2, 0.8, 0.2, 0.1] → R(W) = 0.73
W2 = [1, 0, 0, 0] → R(W) = 1

W1· X = 2
W2· X = 2

Prefers to “spread out” the weights such that each element has the same influence on the prediction

Regularization

- L1 regularization
  - $R(W) = \sum_r \sum_c |W_{r,c}|$

x = [2, 0.1, 0.1, 15]
W1 = [0.2, 0.8, 0.2, 0.1] → R(W) = 1.3
W2 = [1, 0, 0, 0] → R(W) = 1

W1· X = 2
W2· X = 2

Regularization

- L1 regularization
  - $R(W) = \sum_r \sum_c |W_{r,c}|$

x = [2, 0.1, 0.1, 15]
W1 = [0.2, 0.8, 0.2, 0.1] → R(W) = 1.3
W2 = [1, 0, 0, 0] → R(W) = 1

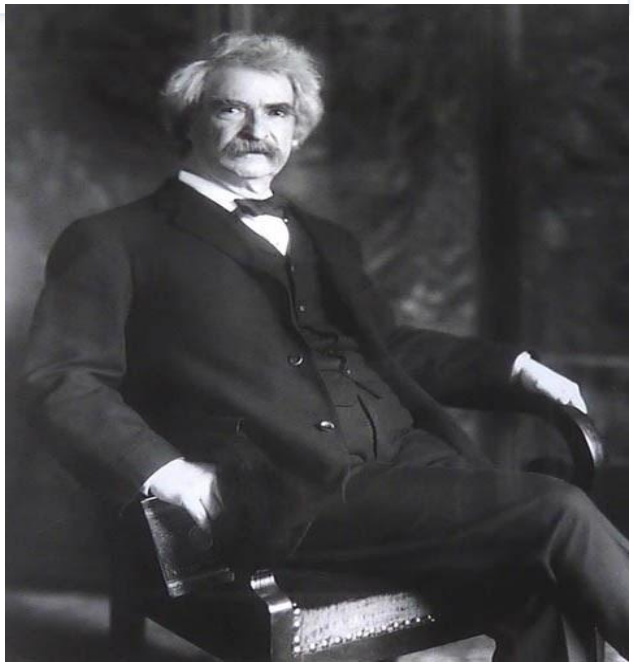
W1· X = 2
W2· X = 2

- Prefers more sparse weight matrices, most of the values in W should be close to 0, except for some values where the weights are allowed to deviate
- Feature selection

Evaluating a classifier

Evaluating a classifier

"*Facts are stubborn things, but statistics are more pliable.*" – **Mark Twain**
"*There are three kinds of lies: lies, damned lies, and statistics.*" – popularized by **Mark Twain**



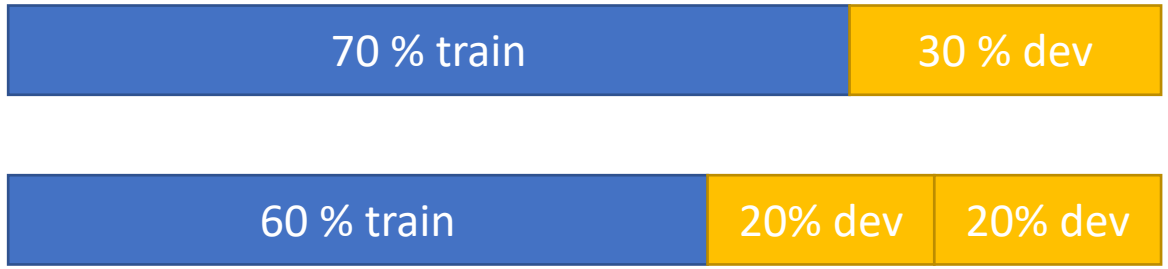
Train, dev and test sets

- Training set
  - Used to train the model, determines what the network learns
- Development (dev) set or validation set
  - Used to tune, iterate, and make decisions: e.g. used to evaluate model performance to guide decisions on hyperparameter settings, compare different models or iterations and select the one that performs best.
- Test set
  - Used to get an unbiased estimate of the final performance of the model

Sometimes is might be ok to not have a test set (only train and dev sets)

How to split your data into train/dev/test set?

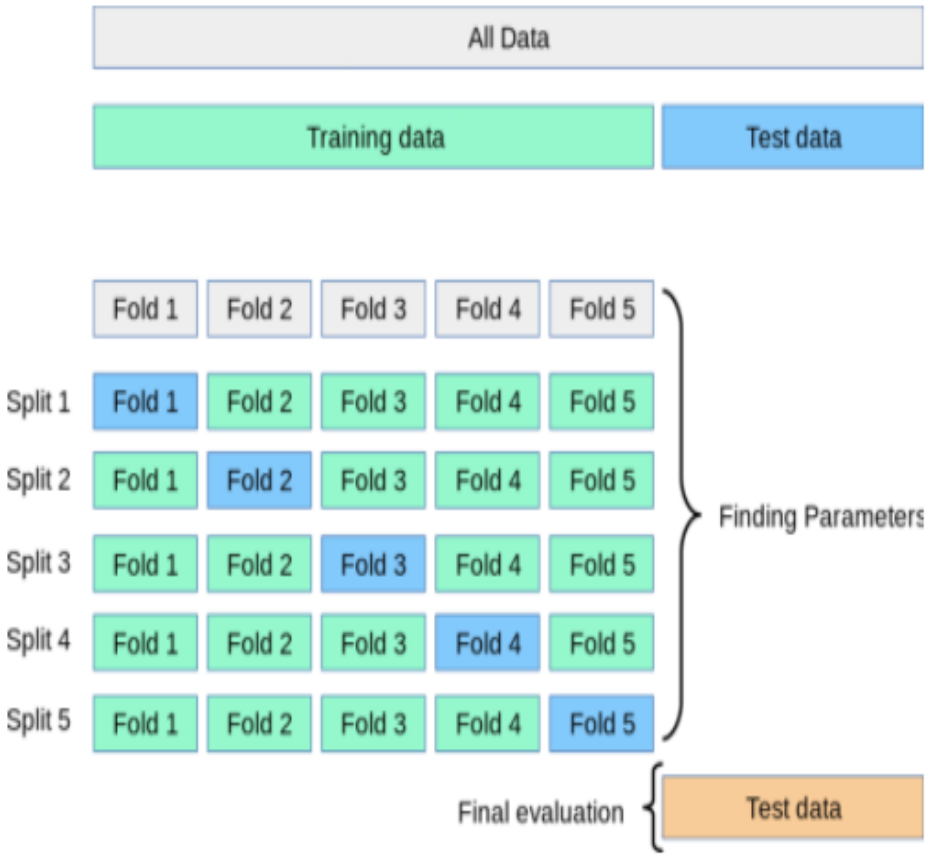
- Before deep learning, when available data was relatively limited (e.g. 10000 images)



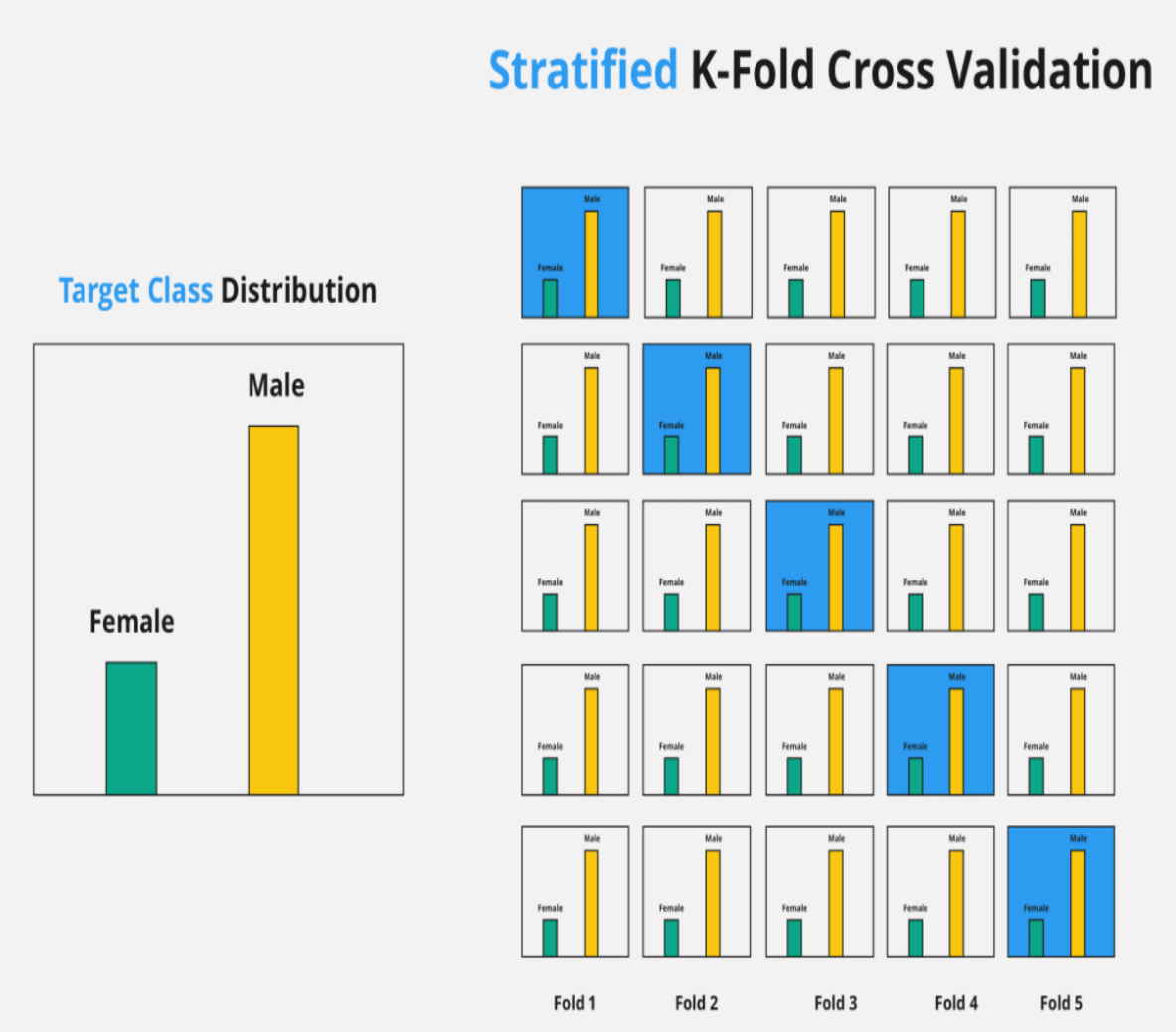
7000 train, 3000 dev

6000 train, 2000 dev, 2000 test

K-fold validation



Stratified k-fold validation



How to split your data into train/dev/test set?

- Large scale datasets
  - ImageNet > 14 million images
  - VGGFace 2 > 3.3 million images

- E.g. 1 million images
  - 980000 train set, 10000 dev set, 10000 test set -> 98% train, 1% dev, 1% test
- E.g. 4 million images
  - 3980000 train set, 10000 dev set, 10000 test set -> 99.5% train, 0.25% dev, 0.25% test

Confusion Terms

TP

True Positive

Predicted this class: actually this class

FP

False Positive

Predicted this class: actually something else

FN

False Negative

Did NOT predict this class: but it actually was

TN

True Negative

Did NOT predict: and it wasn't this class

Accuracy

```
Accuracy = Correct Predictions / Total Predictions
         = (TP + TN) / (TP + TN + FP + FN)
```

!!!Problem!!! If 90% of samples are class A, a model predicting A every time gets 90% accuracy, but is useless.

Binary Precision & Recall

PrecisiON

$$TP / (TP + FP)$$

*The ability to identify **ON**ly relevant samples*
When the model says it's positive, how often is it right?

RecALL

$$TP / (TP + FN)$$

*The ability to identify **ALL** relevant samples*
Out of all the actual positives, how many did the model find?

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

Harmonic mean of precision and recall. It combines both in a single metric

Binary Precision & Recall – tumor classification example

True positive Reality: malignant Model predicted: malignant 1	**False positive** Reality: benign Model predicted: malignant 1
False negative Reality: malignant Model predicted: benign 8	**True negative** Reality: benign Model predicted: benign 90

Binary Precision & Recall – tumor classification example

True positive Reality: malignant Model predicted: malignant 1	**False positive** Reality: benign Model predicted: malignant 1
False negative Reality: malignant Model predicted: benign 8	**True negative** Reality: benign Model predicted: benign 90

Accuracy = (TP + TN)/(TP + TN + FN + FP) = (1 + 90)/(1 + 90 + 8 + 1) = 91/100 = 0.91

Precision = TP / (TP + FP) = 1 / (1 + 1) = 0.5

Recall = TP / (TP + FN) = 1 / (1 + 8) = 0.11

Confusion matrix

		True condition				
	Total population	Condition positive	Condition negative	Prevalence = $\frac{\Sigma \text{ Condition positive}}{\Sigma \text{ Total population}}$	Accuracy (ACC) = $\frac{\Sigma \text{ True positive} + \Sigma \text{ True negative}}{\Sigma \text{ Total population}}$	
Predicted condition	Predicted condition positive	**True positive**	**False positive**, Type I error	Positive predictive value (PPV), Precision = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Predicted condition positive}}$	
	Predicted condition negative	**False negative**, Type II error	**True negative**	False omission rate (FOR) = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Predicted condition negative}}$	Negative predictive value (NPV) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Predicted condition negative}}$	
		True positive rate (TPR), Recall, Sensitivity, probability of detection, Power = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Condition positive}}$	False positive rate (FPR), Fall-out, probability of false alarm = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Condition negative}}$	Positive likelihood ratio (LR+) = $\frac{\text{TPR}}{\text{FPR}}$	Diagnostic odds ratio (DOR) = $\frac{\text{LR+}}{\text{LR−}}$	F_1 score = $2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$
		False negative rate (FNR), Miss rate = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Condition positive}}$	Specificity (SPC), Selectivity, True negative rate (TNR) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Condition negative}}$	Negative likelihood ratio (LR−) = $\frac{\text{FNR}}{\text{TNR}}$		

Image source: https://en.wikipedia.org/wiki/Confusion_matrix

Binary Precision & Recall – tumor classification example

True positive Reality: malignant Model predicted: malignant 1	**False positive** Reality: benign Model predicted: malignant 1
False negative Reality: malignant Model predicted: benign 8	**True negative** Reality: benign Model predicted: benign 90

True positive rate (TPR), Recall, Sensitivity, probability of detection, Power = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Condition positive}}$

False negative rate (FNR), Miss rate = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Condition positive}}$

RECALL family: *Of the real positives, how did the classifier do?*

Recall or Sensitivity or TPR (0.11): *"Of all truly malignant patients, what fraction did I catch?"* Used in medical screening

FNR or Miss Rate (0.89):*"Of all truly malignant patients, what fraction did I miss?"* **FNR = 1 – Recall.**

Binary Precision & Recall – tumor classification example

True positive Reality: malignant Model predicted: malignant 1	**False positive** Reality: benign Model predicted: malignant 1
False negative Reality: malignant Model predicted: benign 8	**True negative** Reality: benign Model predicted: benign 90

False positive rate (FPR), Fall-out,

$$\text{probability of false alarm} = \frac{\Sigma \text{ False positive}}{\Sigma \text{ Condition negative}}$$

Specificity (SPC), Selectivity, True negative rate

$$(\text{TNR}) = \frac{\Sigma \text{ True negative}}{\Sigma \text{ Condition negative}}$$

SPECIFICITY family: *"Of the real negatives, how did the classifier do?"*

Specificity or TNR (= TN / (TN + FP) = 90/91 = 0.989): *"Of all truly benign patients, what fraction did I correctly clear?"*

Many false alarms?

FPR or Fall-out (0.011):*"Of all truly benign patients, what fraction did I wrongly flag as malignant?".* FPR = 1 – Specificity.

Binary Precision & Recall – tumor classification example

True positive Reality: malignant Model predicted: malignant 1	**False positive** Reality: benign Model predicted: malignant 1
False negative Reality: malignant Model predicted: benign 8	**True negative** Reality: benign Model predicted: benign 90

Positive predictive value (PPV), Precision = $\frac{\Sigma \text{ True positive}}{\Sigma \text{ Predicted condition positive}}$	False discovery rate (FDR) = $\frac{\Sigma \text{ False positive}}{\Sigma \text{ Predicted condition positive}}$

PRECISION family: "When the classifier makes a positive prediction, how trustworthy is it? "

Precision or PPV (0.50): *"Of every patient I flagged as malignant, how many actually were?"*

FDR (0.50): *"Of every patient I flagged as malignant, how many were actually benign?".* FDR = 1 − Precision.

Binary Precision & Recall – tumor classification example

True positive Reality: malignant Model predicted: malignant 1	**False positive** Reality: benign Model predicted: malignant 1
False negative Reality: malignant Model predicted: benign 8	**True negative** Reality: benign Model predicted: benign 90

False omission rate (FOR) = $\frac{\Sigma \text{ False negative}}{\Sigma \text{ Predicted condition negative}}$

Negative predictive value (NPV) = $\frac{\Sigma \text{ True negative}}{\Sigma \text{ Predicted condition negative}}$

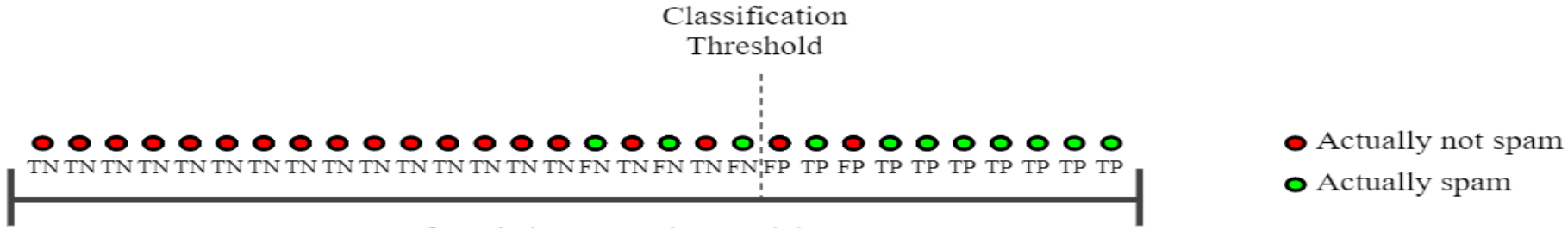
NPV family: *"When the classifier makes a negative prediction, how trustworthy is it?*

NPV (=90/98=0.918): *"Of every patient I cleared as benign, how many actually were benign?"* Are negative calls reliable?

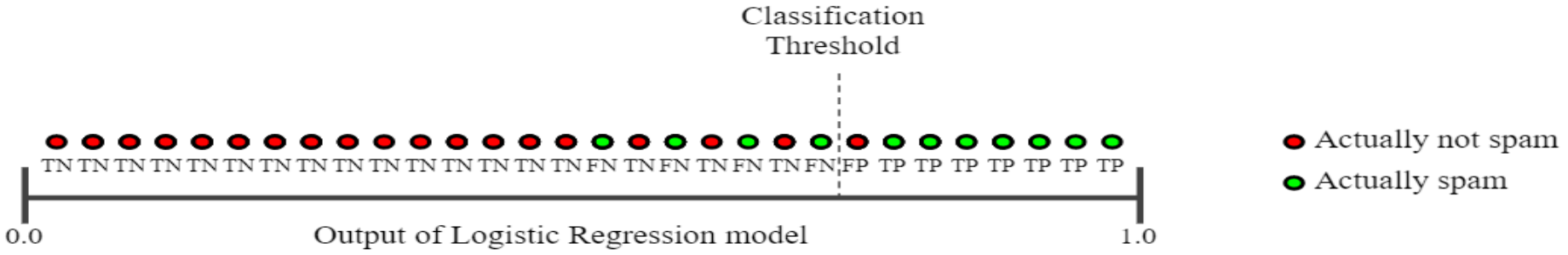
FOR (0.082): *"Of every patient I cleared as benign, how many were actually malignant?".*

These are the dangerous silent misses. FOR = 1 − NPV.

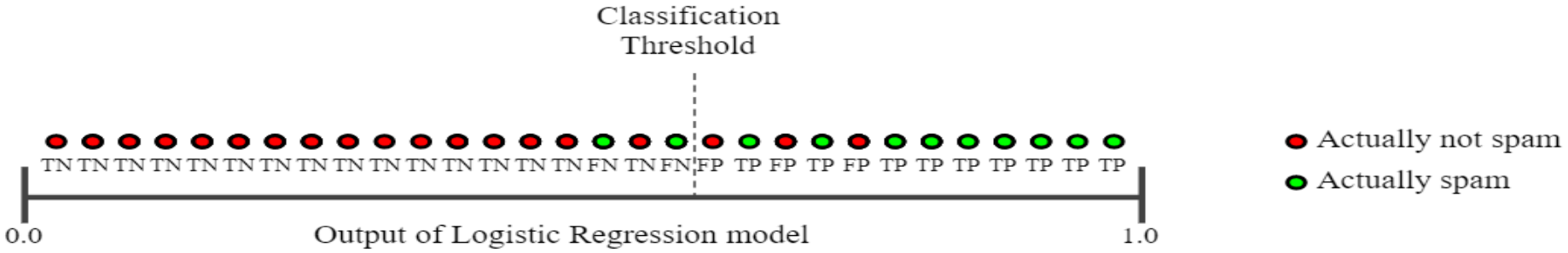
Precision vs. recall



Precision: 0.8
Recall: 0.73



Precision: 0.88
Recall: 0.64



Precision: 0.75
Recall: 0.82

https://developers.google.com/machine-learning/crash-course/classification/precision-and-recall

Precision vs. recall – when to favour one vs the other?

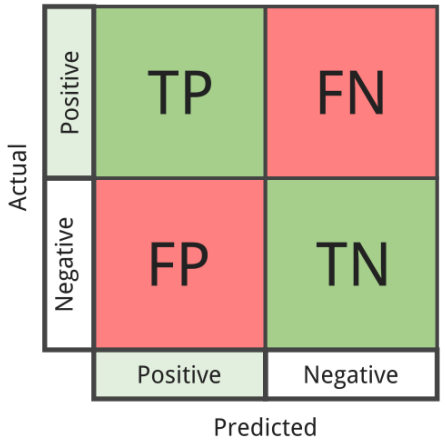
- *Precision*: spam detection, predict when to launch a satellite, pregnancy tests etc.
- *Recall*: airport security (make sure that every suspicions event is investigated), cancer prediction, detecting credit card frauds etc.

Receiver Operating characteristic curve

ROC curve

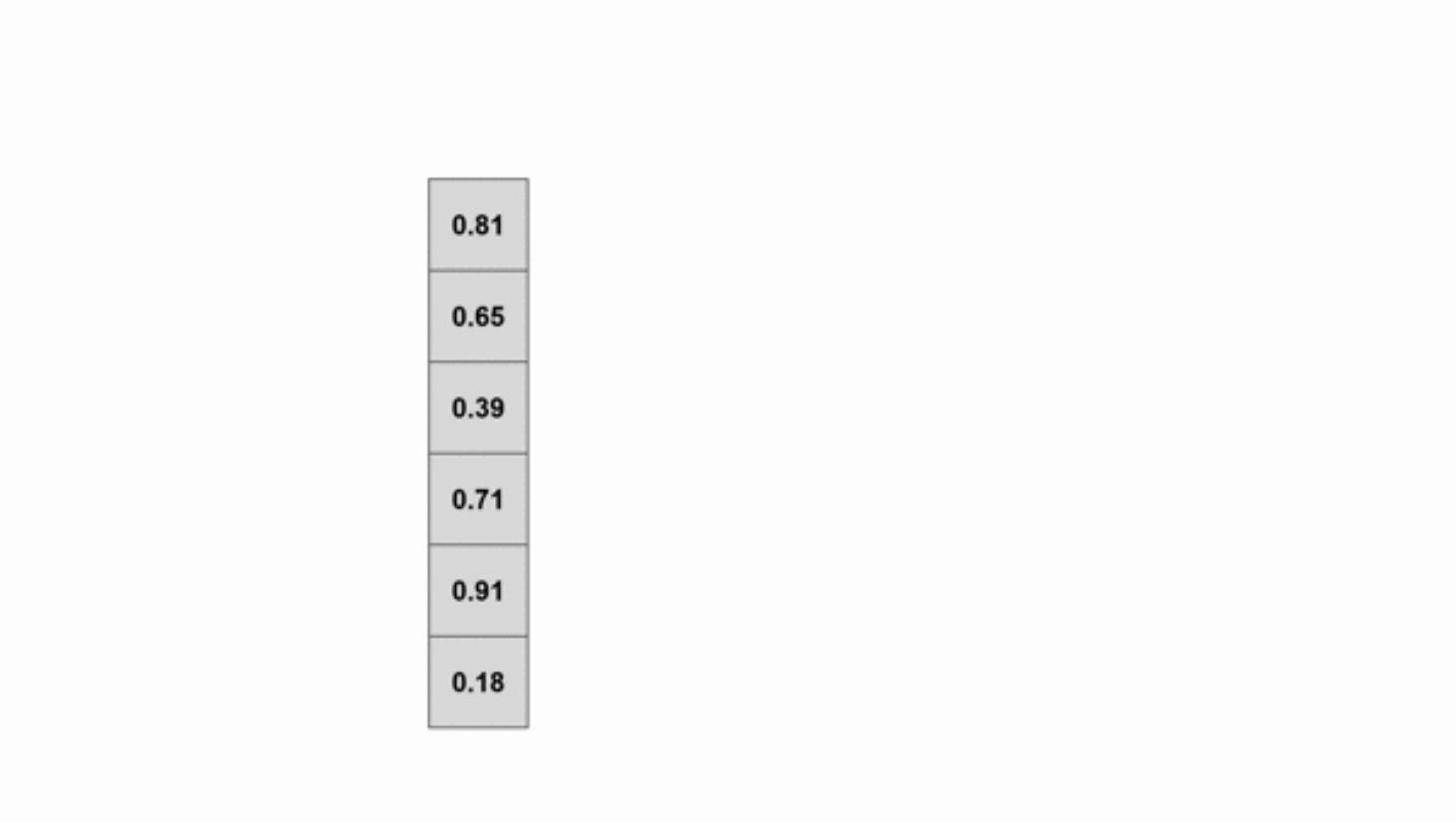
- Shows the performance of a classifier at different classification thresholds

False Positive Rate (1- specificity) = FP / (FP + TN) – x axis
True Positive Rate (sensitivity) = TP / (TP + FN) – y axis



Receiver Operating characteristic curve

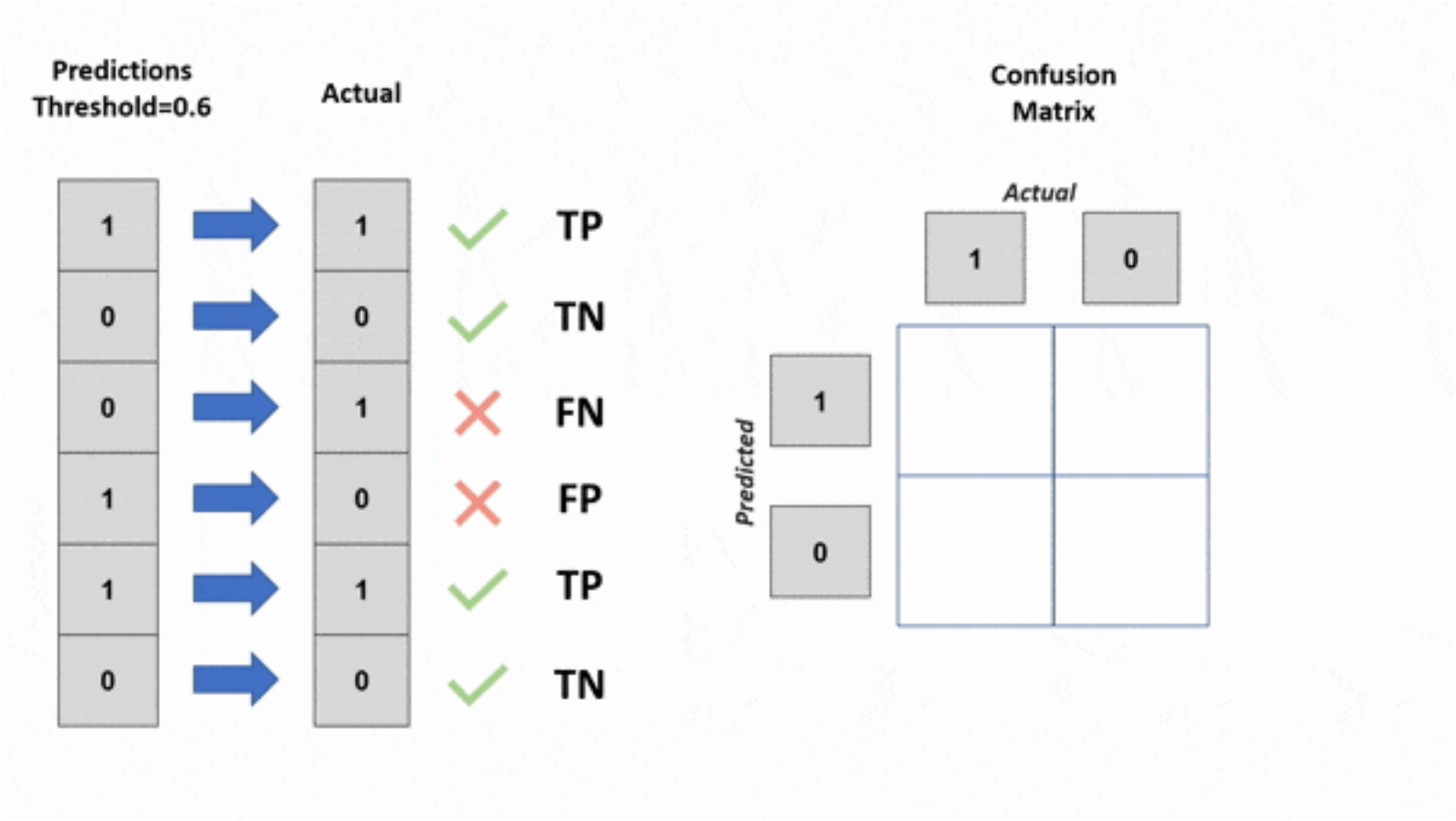
ROC curve



https://towardsdatascience.com/roc-curve-and-auc-from-scratch-in-numpy-visualized-2612bb9459ab

Receiver Operating characteristic curve

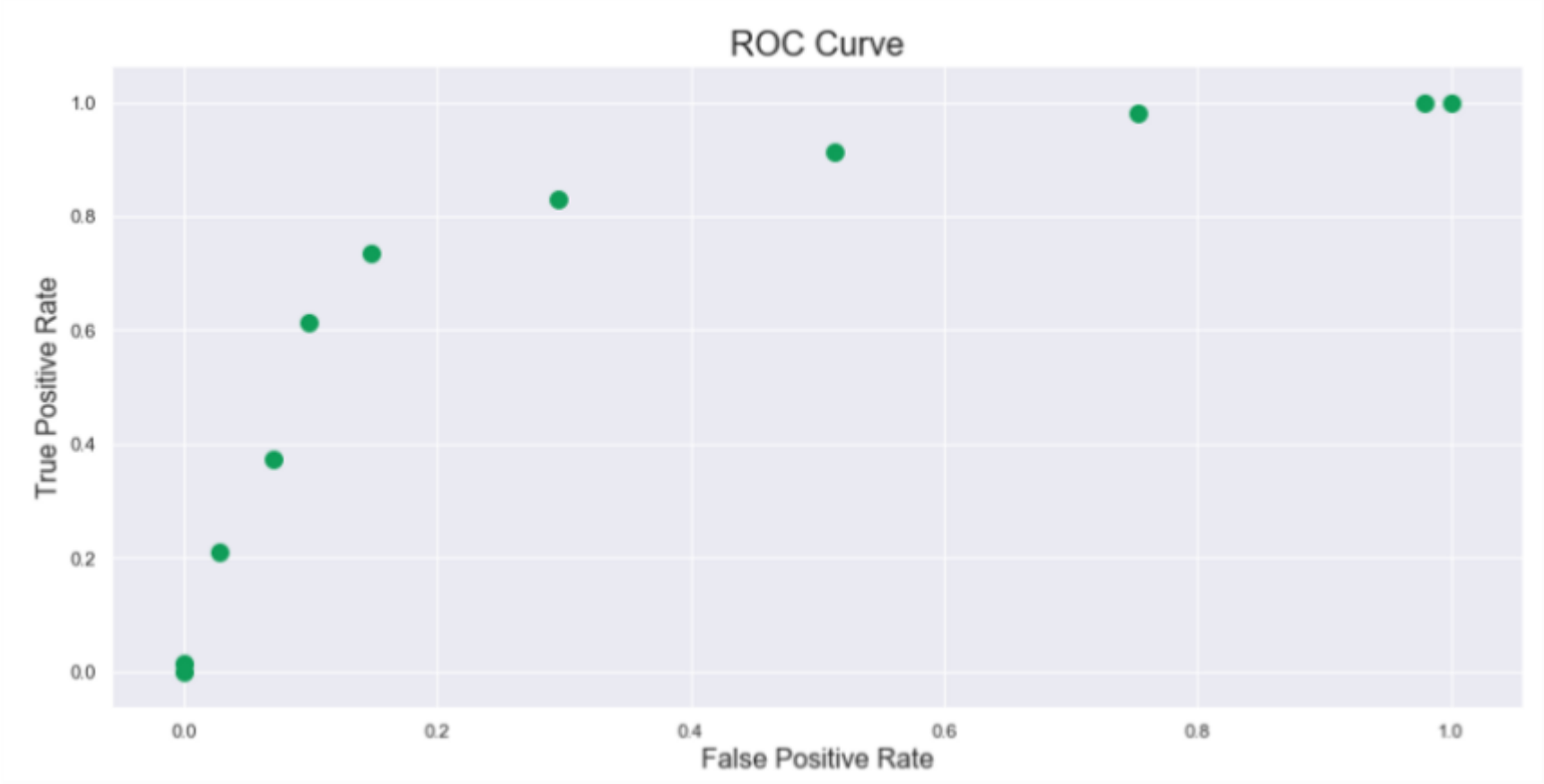
ROC curve



https://towardsdatascience.com/roc-curve-and-auc-from-scratch-in-numpy-visualized-2612bb9459ab

Receiver Operating characteristic curve

ROC curve

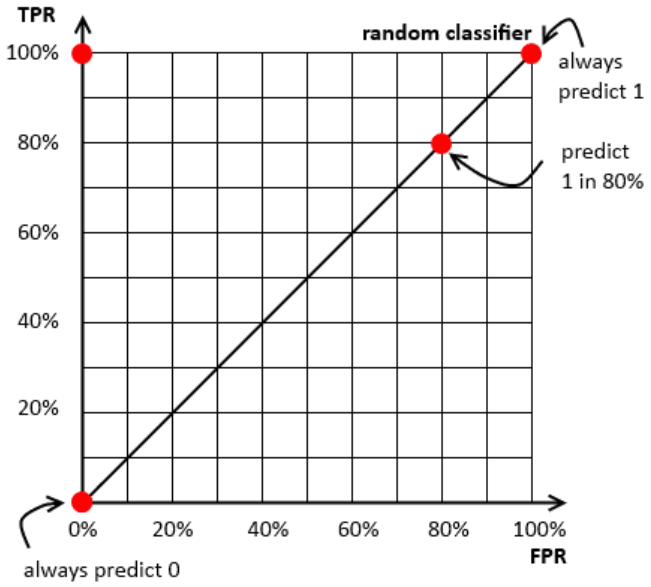


https://towardsdatascience.com/roc-curve-and-auc-from-scratch-in-numpy-visualized-2612bb9459ab

Receiver Operating characteristic curve

ROC curve

- Shows the performance of a classifier at different classification thresholds

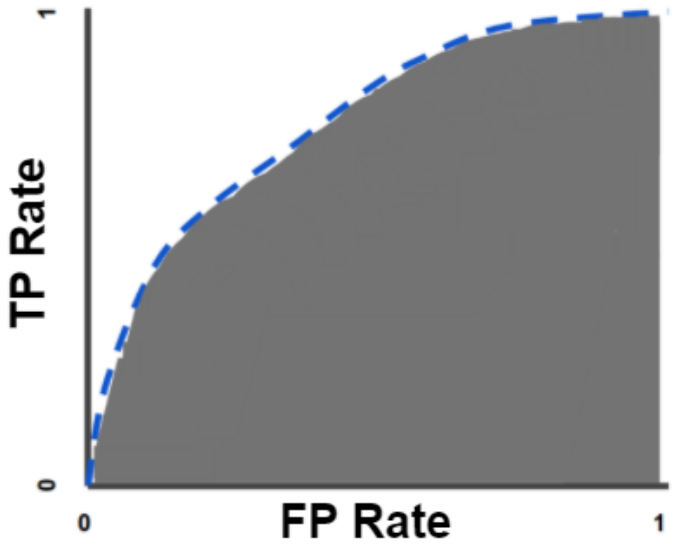


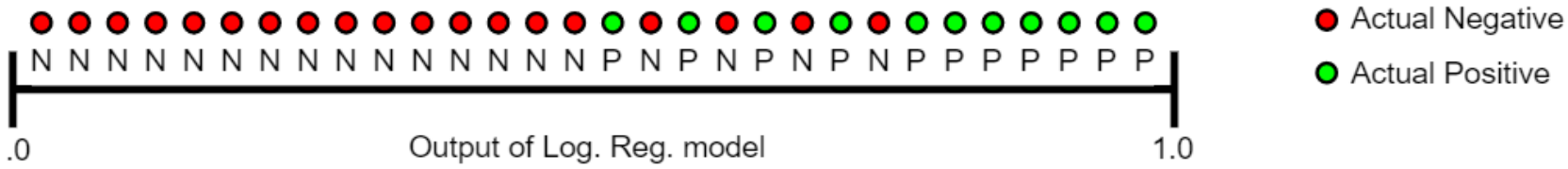
False Positive Rate (1- specificity) = FP / (FP + TN) – x axis
True Positive Rate (sensitivity) = TP / (TP + FN) – y axis

Receiver Operating characteristic curve

Area under the curve

- Measures the entire two-dimensional area underneath the entire ROC curve
- Makes it easier to compare one ROC curve to another one





Multi-class settings

- In multi-class settings, we don't just have positive and negative.
- We evaluate each class individually as if it were a binary problem.

Macro average	Micro average
Calculate metric per class, then average them.	Aggregate all TP, FP, FN first, then calculate.
Treats every **class** as equally important.	Treats every **sample** as equally important.
Useful for imbalanced data where the small class is important.	General performance when class size varies.

Multi-class settings
Example

			Predicted	
	Label	**Airplane**	**Boat**	**Car**
	Airplane	**2**	1	0
Actual	**Boat**	0	1	0
	Car	1	2	**3**

TP? FP? FN?

https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Multi-class settings
Example

			Predicted	
	Label	**Airplane**	**Boat**	**Car**
	Airplane	**2**	1	0
Actual	**Boat**	0	1	0
	Car	1	2	**3**

Airplane: TP = 2, FP = 1, FN = 1
Boat: TP = 1, FP = 3, FN = 0
Car: TP = 3, FP = 0, FN = 3

https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Multi-class settings
Example

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Precision	Recall	F1 Score
Airplane	2	1	1	0.67	0.67	2 * (0.67 * 0.67) / (0.67 + 0.67) = **0.67**
Boat	1	3	0	0.25	1.00	2*(0.25 * 1.00) / (0.25 + 1.00) = **0.40**
Car	3	0	3	1.00	0.50	2 * (1.00 * 0.50) / (1.00 + 0.50) = **0.67**

Airplane: TP = 2, FP = 1, FN = 1
Precision = 2/3 ~ 0.67
Recall = 2/3 ~ 0.67

Boat: TP = 1, FP = 1, FN = 0
Precision = 1/4 = 0.25
Recall = 1/1 = 1

https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Multi-class settings
Macro average

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Precision	Recall	F1 Score
Airplane	2	1	1	0.67	0.67	2 * (0.67 * 0.67) / (0.67 + 0.67) = **0.67**
Boat	1	3	0	0.25	1.00	2*(0.25 * 1.00) / (0.25 + 1.00) = **0.40**
Car	3	0	3	1.00	0.50	2 * (1.00 * 0.50) / (1.00 + 0.50) = **0.67**

Calculate metric per class, then average them.

Precision = (0.67 + 0.25 + 1)/3 = 1.92/3 = 0.64

Recall = (0.67 + 1 + 0.5)/3 = 0.72

F1 = (0.67 + 0.4 + 0.67)/3 = 1.74/3 = 0.58

https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Multi-class settings
Weighted macro average

Label	Per-Class F1 Score	Support	Support Proportion	Weighted Average F1 Score
Airplane	0.67	3	0.3	$(0.67 * 0.3) + (0.40 * 0.1) + (0.67 * 0.6)$ **= 0.64**
Boat	0.40	1	0.1	
Car	0.67	6	0.6	
Total	-	10	1.0	

https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Multi-class settings
Micro average

Label	True Positive (TP)	False Positive (FP)	False Negative (FN)	Micro-Averaged Values
Airplane	2	1	1	Precision = $\frac{6}{6+4}$ = **0.60**
Boat	1	3	0	Recall = $\frac{6}{6+4}$ = **0.60**
Car	3	0	3	F1 Score = $\frac{6}{6+\frac{1}{2}(4+4)}$ = **0.60**
TOTAL	**6**	**4**	**4**	

We first sum the respective TP, FP, and FN values across **all classes** and then plug them into the equations to get the score.

The **proportion** of **correctly classified** observations out of all observations

Precision = TP / (TP + FP) = 6 / (6 + 4) = 0.6

Recall = TP/(TP + FN) = 6 / (6 + 4) = 0.6

F1 = 2*(Precision * Recall) / (Precision + Recall) = 2 * 0.36/1.2 = 0.6

https://iamirmasoud.com/2022/06/19/understanding-micro-macro-and-weighted-averages-for-scikit-learn-metrics-in-multi-class-classification-with-example/

Optimization

How can we find the best value for W?

Gradient descent

Follow the slope!

Follow the slope

- In 1D the derivative of a function is:
  - Slope of the “line”

$$f'^{(a)} = \frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

Follow the slope

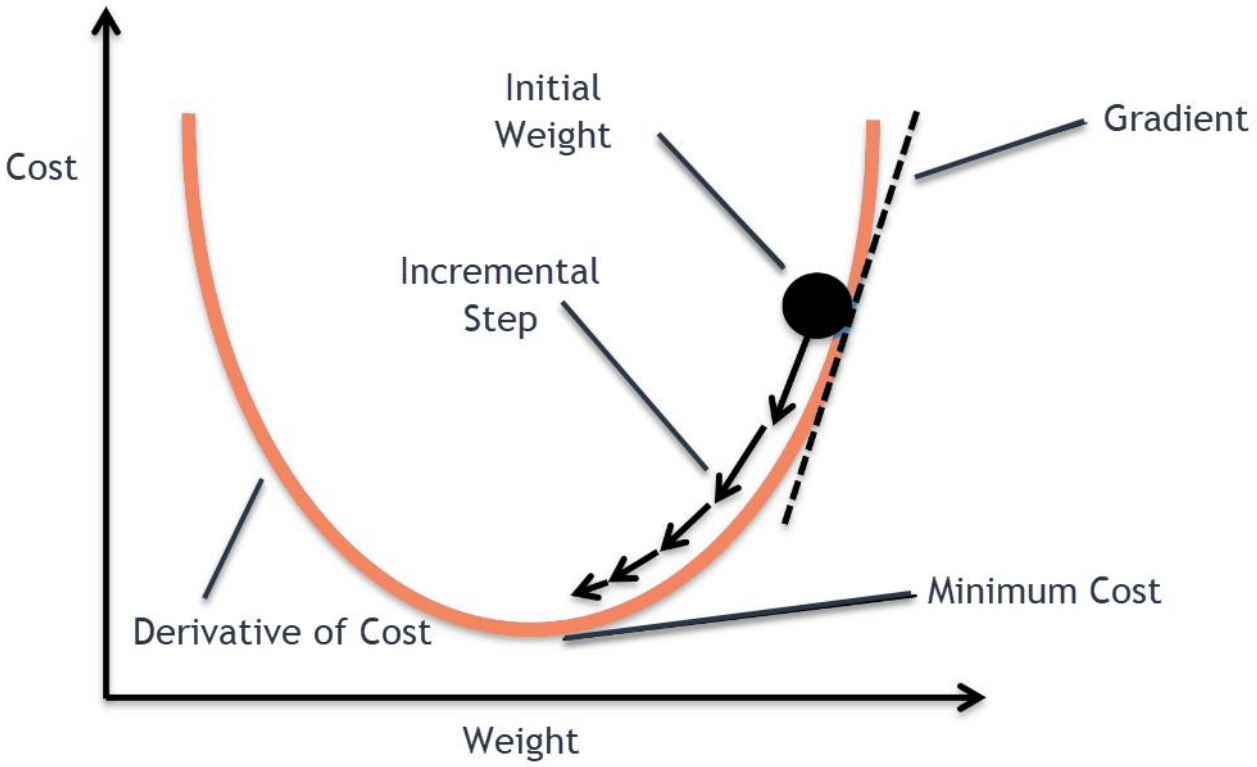


Image source: https://medium.com/analytics-vidhya/stochastic-gradient-descent-1ab661fabf89

Follow the slope

- In N dimensions, the gradient is a vector containing the partial derivative along each dimensions

Numerical gradient

w	[0.3, 0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]
	loss 1.2345
gradient	?

Numerical gradient

W [0.3,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

loss 1.2345

W + h [0.3001,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

h = 0.0001

loss 1.2331

= 0.3 + h

= 0.3 + 0.0001

$$f'(a) = \frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}$$

gradient ?

Numerical gradient

W [0.3,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,...]

loss 1.2345

W + h [0.3001,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,...]

h = 0.0001

loss 1.2331

= 0.3 + h

= 0.3 + 0.0001

gradient [(1.2345 - 1.2331)/0.0001, ...]

Numerical gradient

W [0.3,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

loss 1.2345

W + h [0.3001,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

h = 0.0001

loss 1.2331

= 0.3 + h

= 0.3 + 0.0001

gradient [23.5, ….]

Numerical gradient

W [0.3,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

loss 1.2345

W + h [0.3, 0.2401, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

h = 0.0001

loss 1.2345

= 0.24 + h

gradient [23.5, …]

Numerical gradient

W [0.3,0.24, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

loss 1.2345

W + h [0.3001,0.2401, -1.9, 0.22, 0.3, 0.9, -1.7, 1.5, 0.29,…]

h = 0.0001

loss 1.2345

= 0.24 + h

gradient [23.5, (1.2345 – 1.2345)/0.0001, ….]

Analytical gradient

- Numerical gradient: slow and approximate
- **Analytical gradient.** Given:

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i\left(x_i, y_i, W\right) + \lambda R(W) \qquad L_i = \sum_{j\,!=y_i} \max(0, s_j - s_{y_i} + m)$$

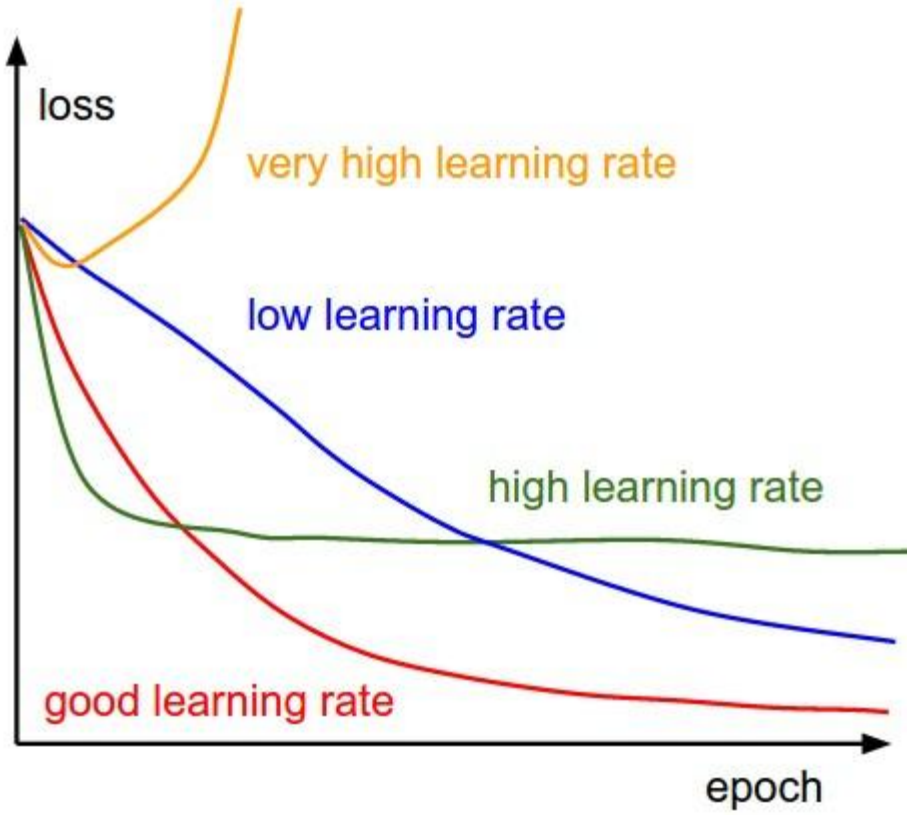
, we need to compute

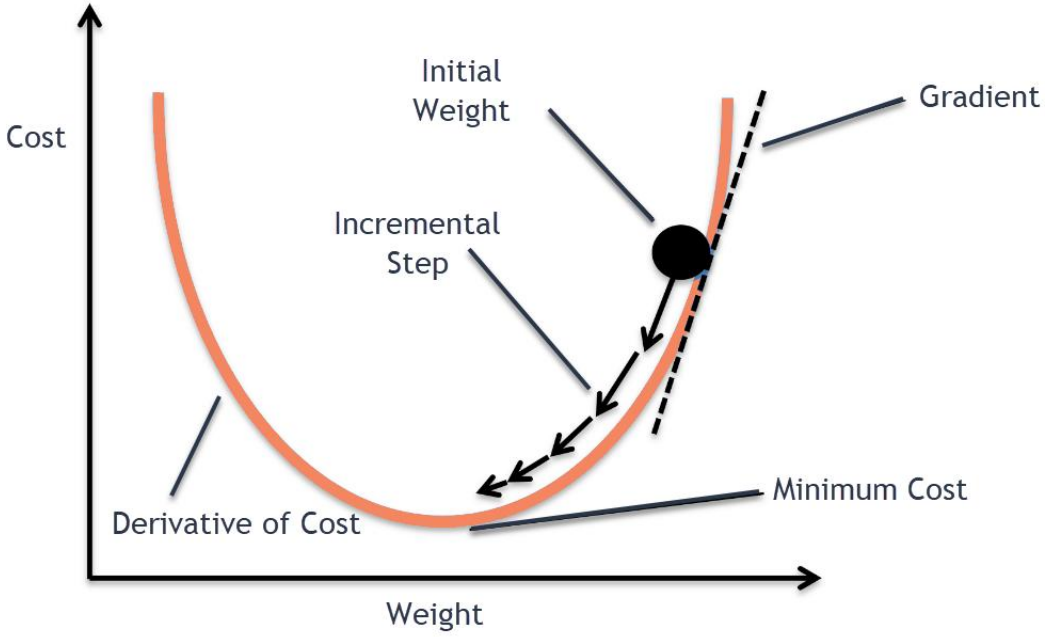
$$\frac{\partial L}{\partial W} = \frac{1}{N}\sum_{1}^{N} \frac{\partial L_i(x_i, y_i, W)}{\partial W} + \lambda \frac{\partial R(W)}{\partial W}$$

and we can use calculus for this!

Gradient descent

```
while True:
    W_gradient = compute_gradient(loss_func, data, W)
    # update parameters
    W += -lr*W_gradient
```





Example adapted from: https://cs231n.github.io/

Recommended resources

- https://cs231n.github.io/linear-classify/#intro
- Loss functions (Andrew Ng`s great explanations):
  - https://www.youtube.com/watch?v=LLux1SW--oM
  - https://www.youtube.com/watch?v=ueO_Ph0Pyqk
- www.coursera.org – Deep Learning specialization: Neural Networks and Deep Learning (week 1 and 2)